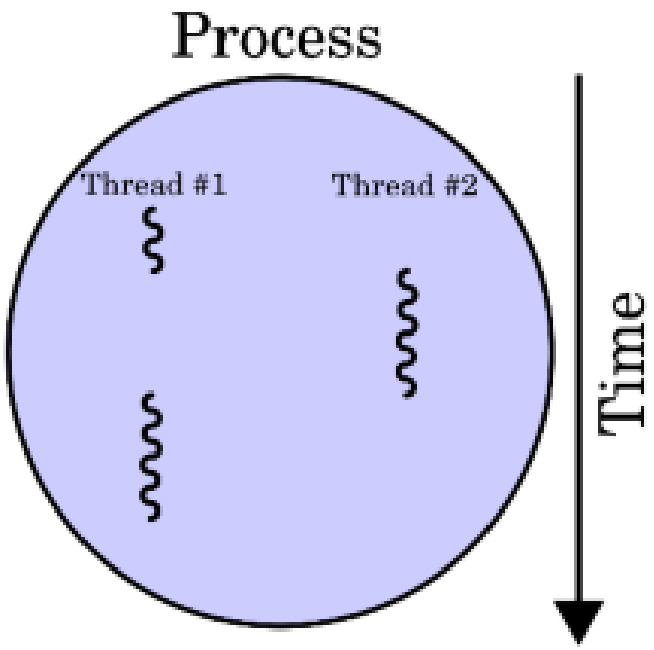
Chatper04-Cloud Programming Models

By: Diwakar Upadhyaya

Thread Programming

- With computer programming, a thread is a small set of instructions designed to be scheduled and executed by the CPU independently of the parent process.
- For example, a program may have an open thread waiting for a specific event to occur or running a separate job, allowing the main program to perform other tasks.

A process with two threads of execution, running on a single processor.



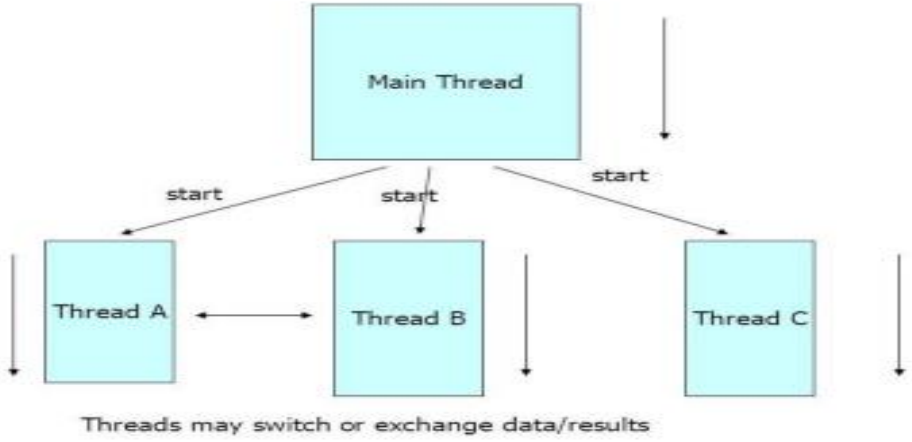
- Multithreading is becoming increasingly important, both as a program structuring mechanism and to support efficient parallel computations.
- The research in analysis for multithreaded programs, focusing on ways to improve the efficiency of analyzing interactions between threads and to detect data races.
- We try to compare the results of the sequential and threaded programming so that we can deploy these models to cloud platforms

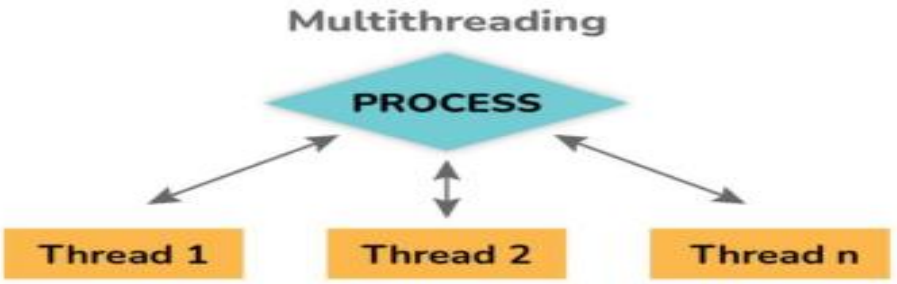
- Multithreading is a widely used structuring technique for modern software.
- Programmers use multiple threads of control for a variety of reasons: to build responsive servers that interact with multiple clients,

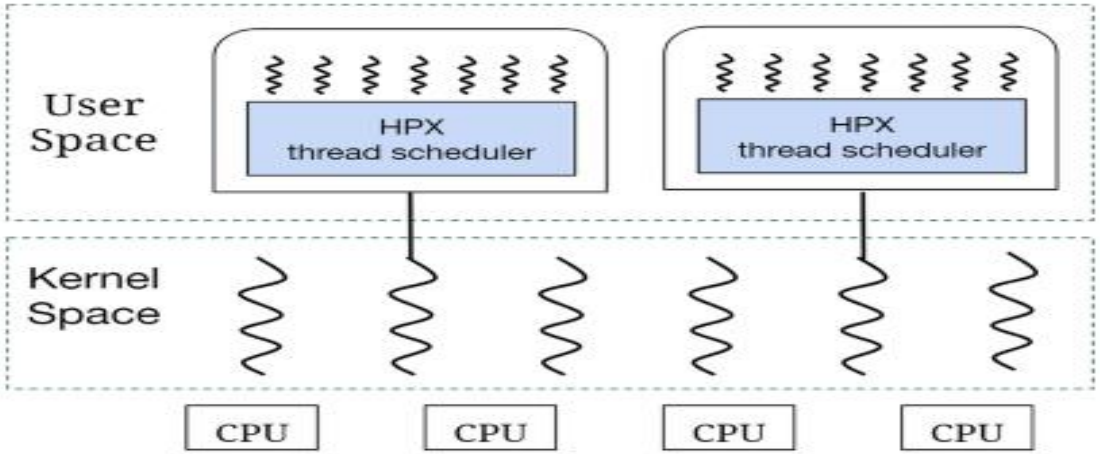
  to run computations in parallel on a multiprocessor for performance,

  and as a structuring mechanism for implementing rich user interfaces.
- In general, threads are useful whenever the software needs to manage a set of tasks with varying interaction latencies, exploit multiple physical resources, or execute largely independent tasks in response to multiple external events.

- Developing parallel applications requires an understanding of the problem and its logical structure.
- Understanding the dependencies and the correlation of tasks within an application is fundamental for designing the right program structure and to introduce parallelism where appropriate.
- Decomposition is a useful technique that helps to understand whether a problem is divided into components (or tasks) that can be executed concurrently.


Main Thread
start
start
start
Thread A
Thread B
Thread C
Threads may switch or exchange data/results


Multithreading
PROCESS
Thread 1
Thread 2
Thread n


User Space
HPX thread scheduler
HPX thread scheduler
Kernel Space
CPU
CPU
CPU
CPU

User-level thread:

- They are created and managed by users. They are used at the application level.
- The operating system does not recognize the user-level thread. User threads can be easily implemented and it is implemented by the user.
- If a user performs a user-level thread blocking operation, the whole process is blocked.
- The kernel level thread does not know nothing about the user level thread.

kernel thread

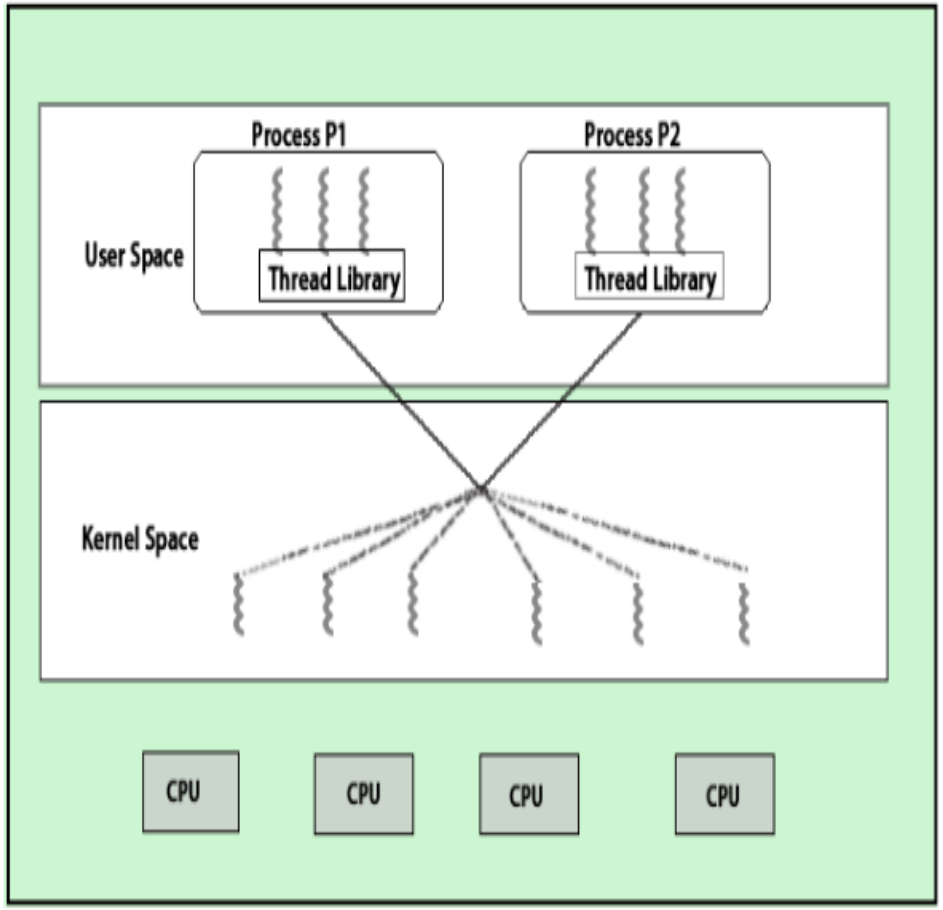
- The kernel thread recognizes the operating system.
- There is a thread control block and process control block in the system for each thread and process in the kernel-level thread.
- The kernel-level thread is implemented by the operating system. The kernel knows about all the threads and manages them.
- The kernel-level thread offers a system call to create and manage the threads from user- space.
- o The implementation of kernel threads is more difficult than the user thread.
- Context switch time is longer in the kernel thread.

BASIS FOR COMPARISON	MULTIPROCESSING	MULTITHREADING
Basic	Multiprocessing adds CPUs to increase computing power.	Multithreading creates multiple threads of a single process to increase computing power.
Execution	Multiple processes are executed concurrently.	Multiple threads of a single process are executed concurrently.
Creation	Creation of a process is time-consuming and resource intensive.	Creation of a thread is economical in both sense time and resource.
Classification	Multiprocessing can be symmetric or asymmetric.	Multithreading is not classified.

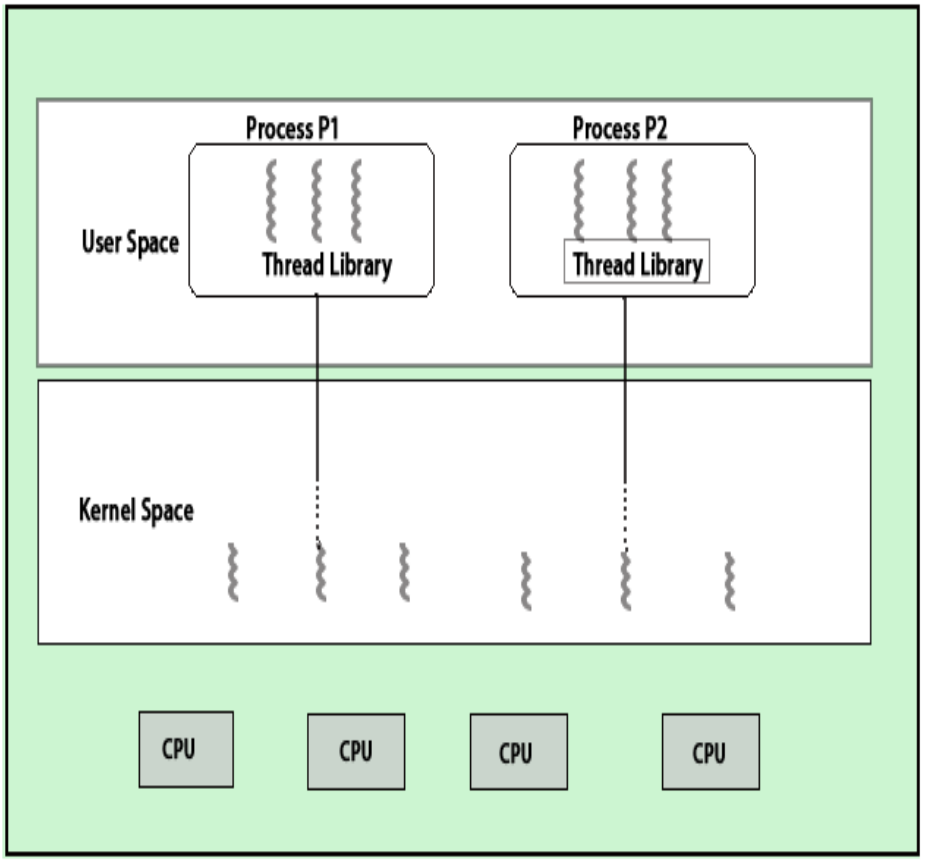
Multithreading Models

These models are of three types

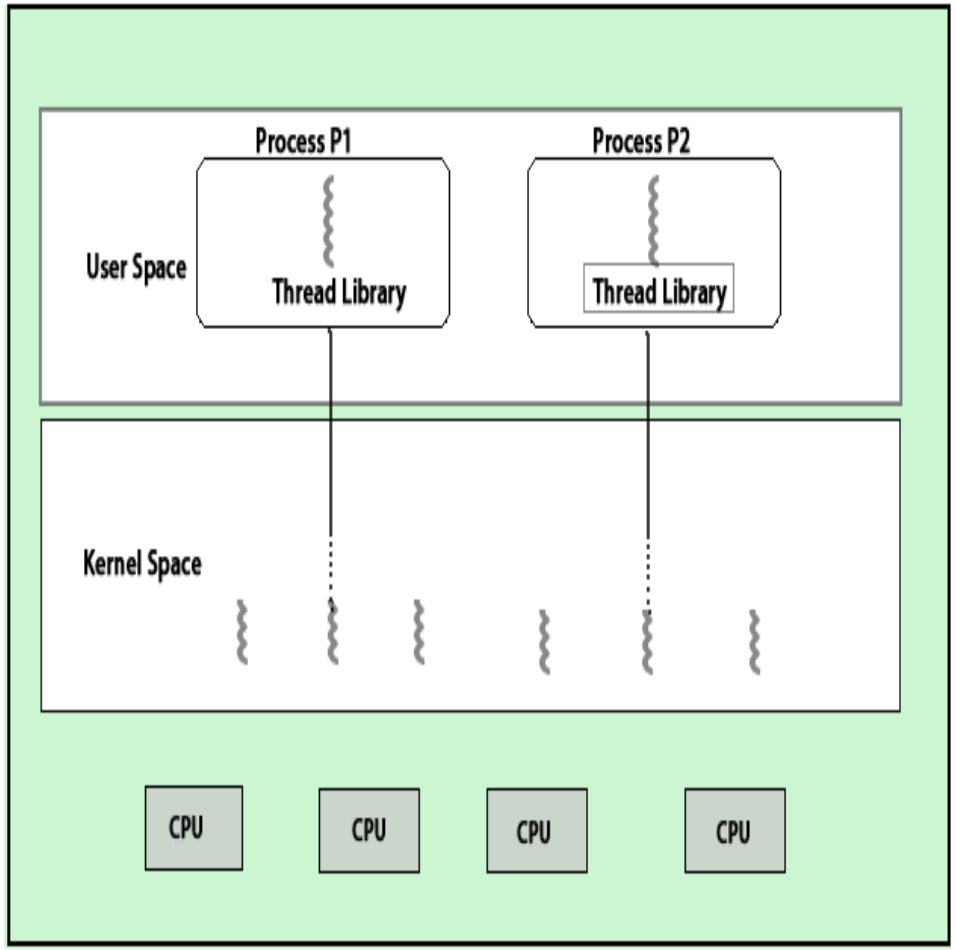
- Many to many
- Many to one
- One to one



- **Many to many:** Any number of user threads can interact with an equal or lesser number of kernel threads.



- **Many to one:** It maps many user-level threads to one Kernel-level thread.



- **One to one:** Relationship between the user-level thread and the kernel-level thread is one to one.

Uses of Multithreading

- It is a way to introduce parallelism in the system or program.
- So, we can use it anywhere we see parallel paths (where two threads are not dependent on the result of one another) to make it fast and easy.
- For example: Processing of large data where it can be divided into parts and get it done using multiple threads.
- Applications which involve mechanism like validate and save, produce and consume, read and validate are done in multiple threads.
- Few examples of such applications are online banking, recharges, etc.

- It can be used to make games where different elements are running on different threads.
- In Android, it is used to hit the APIs which are running in the background thread to save the application from stopping.
- In web applications, it is used when we want our app to get asynchronous calls and perform asynchronously.

Advantages of Multithreading

Below are mentioned some of the advantages:

- **Economical**: It is economical as they share the same processor resources. It takes lesser time to create threads.
- **Resource sharing**: It allows the threads to share resources like data, memory, files, etc. Therefore, an application can have multiple threads within the same address space.
- **Responsiveness**: It increases the responsiveness to the user as it allows the program to continue running even if a part of it is performing a lengthy operation or is blocked.
- **Scalability**: It increases parallelism on multiple CPU machines. It enhances the performance of multi-processor machines. It makes the usage of CPU resources better.

Why should we use Multithreading?

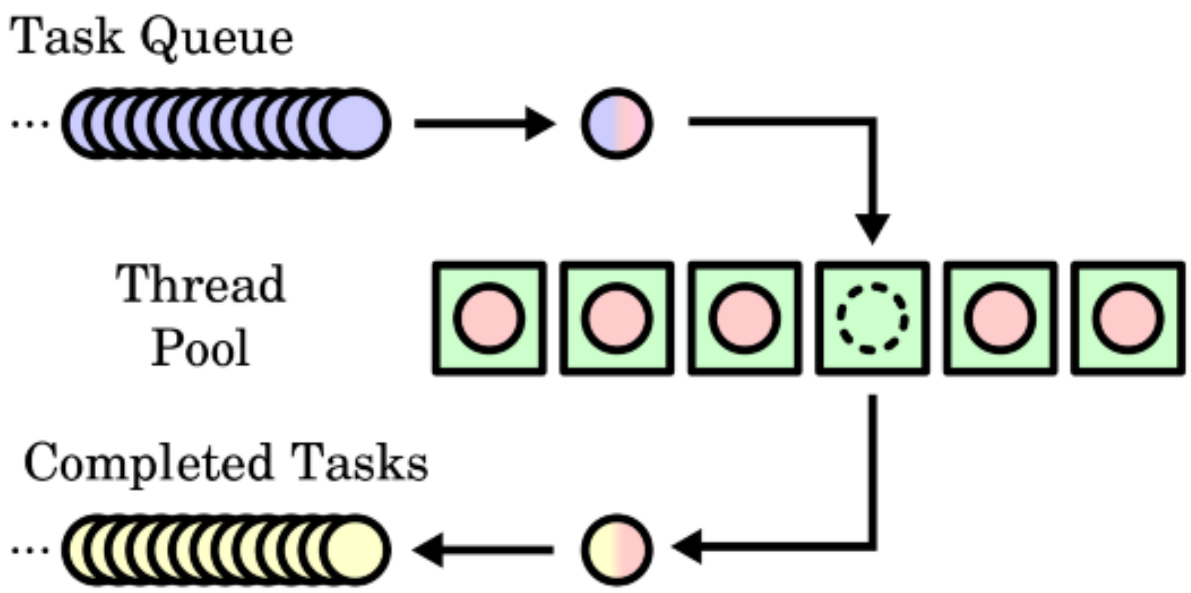
We should use this because of the following reasons:

- To increase parallelism
- To make most of the available CPU resources.
- To improve application responsiveness and give better interaction with the user.

Task Programming

- Task computing is a wide area of distributed system programming encompassing several different models of architecting distributed applications
- A task represents a program, which require input files and produce output files as a result of its execution. Applications are then constituted of a collection of tasks.
- These are submitted for execution and their output data are collected at the end of their execution.
- This chapter characterizes the abstraction of a task and provides a brief overview of the distributed application models that are based on the task abstraction.
- The Aneka Task Programming Model is taken as a reference implementation to illustrate the execution of bag-of-tasks (BoT) applications on a distributed infrastructure.

- In computer programming, a task is a basic unit of programming that an operating system controls.
- All of today's widely-used operating systems support multitasking, which allows multiple tasks to run concurrently, taking turns using the resources of the computer.
- Depending on how the operating system defines a task in its design, this unit of programming may be an entire program or each successive invocation of a program.
- In computing, a task is a unit of execution or a unit of work. The term is ambiguous; precise alternative terms include process, light-weight process, thread (for execution), step, request, or query (for work).


Task Queue
Thread
Pool
Completed Tasks

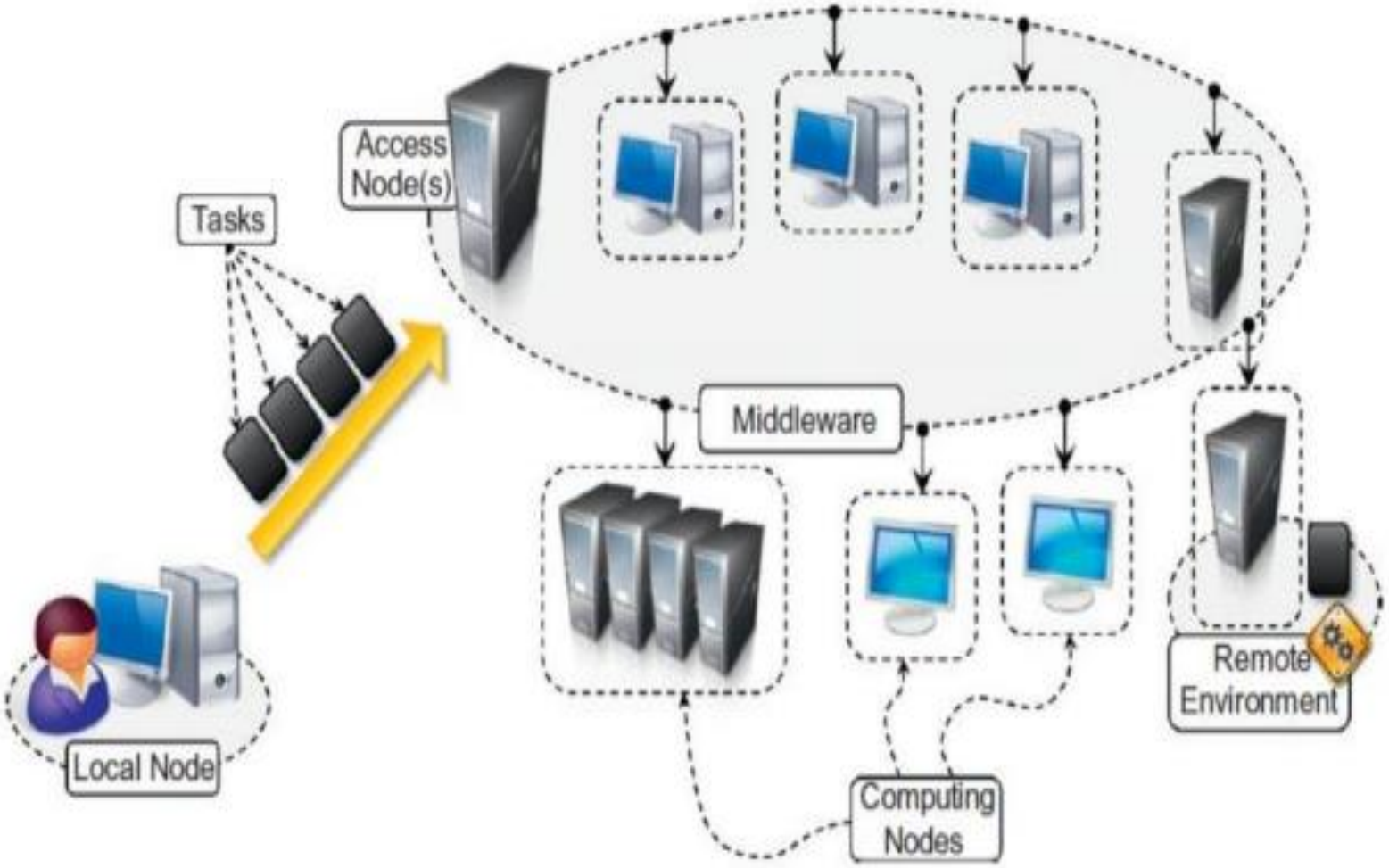
- The Task Programming Model is a high-level multithreaded programming model.
- It is designed to allow Maple code to be written that takes advantage of multiple processors, while avoiding much of the complexity of traditional multithreaded programming.
- No explicit threading, users create Tasks, not Threads.
- Task computing is a wide area of distributed system programming encompassing several different models of architecting distributed applications, which, eventually, are based on the same fundamental abstraction: the task.
- Applications are then constituted of a collection of tasks.

Task Computing

- A task identifies one or more operations that produce a distinct output and that can be isolated as a single logical unit.
- In practice, a task is represented as a distinct unit of code, or a program, that can be separated and executed in a remote run time environment.

- Multithreaded programming is mainly concerned with providing a support for within a single machine.
- Task computing provides distribution by harnessing the compute power of several computing nodes. Hence, the presence of a distributed infrastructure is explicit in this model.
- Now clouds have emerged as an attractive solution to obtain a huge computing power on demand for the execution of distributed applications.
- To achieve it, suitable middleware is needed. A reference scenario for task computing is depicted in Figure

CLOUD COMPUTING



- The middleware is a software layer that enables the coordinated use of multiple resources, which are drawn from a data center or geographically distributed networked computers.
- A user submits the collection of tasks to the access point(s) of the middleware, which will take care of scheduling and monitoring the execution of tasks.
- Each computing resource provides an appropriate runtime environment.
- Task submission is done using the APIs provided by the middleware, whether a Web or programming language interface

- Appropriate APIs are also provided to monitor task status and collect their results upon completion.
- It is possible to identify a set of common operations that the middleware needs to support the creation and execution of task-based applications.

These operations are:

- Coordinating and scheduling tasks for execution on a set of remote nodes
- Moving programs to remote nodes and managing their dependencies
- Creating an environment for execution of tasks on the remote nodes
- Monitoring each task's execution and informing the user about its status
- Access to the output produced by the task.

Frameworks for task computing

- Some popular software systems that support the task-computing framework are:

1. Condor
2. Globus Toolkit
3. Sun Grid Engine (SGE)
4. BOINC

- Architecture of all these systems is similar to the general reference architecture depicted in Figure
- They consist of two main components: a scheduling node (one or more) and worker nodes.
- The organization of the system components may vary.

1. Condor

- Condor is the most widely used and long-lived middleware for managing clusters, idle workstations, and a collection of clusters.
- Condor supports features of batch-queuing systems along with the capability to checkpoint jobs and manage overload nodes.
- It provides a powerful job resource-matching mechanism, which schedules jobs only on resources that have the appropriate runtime environment.
- Condor can handle both serial and parallel jobs on a wide variety of resources.
- It is used by hundreds of organizations in industry, government, and academia to manage infrastructures

2. Globus Toolkit

- The Globus Toolkit is a collection of technologies that enable grid computing.
- It provides a comprehensive set of tools for sharing computing power, databases, and other services across corporate, institutional, and geographic boundaries.
- The toolkit features software services, libraries, and tools for resource monitoring, discovery, and management as well as security and file management.
- The toolkit defines a collection of interfaces and protocol for interoperation that enable different systems to integrate with each other and expose resources outside their boundaries.

3. Sun Grid Engine (SGE)

- Sun Grid Engine (SGE), now Oracle Grid Engine, is middleware for workload and distributed resource management.
- Initially developed to support the execution of jobs on clusters, SGE integrated additional capabilities and now is able to manage heterogeneous resources and constitutes middleware for grid computing.
- It supports the execution of parallel, serial, interactive, and parametric jobs and features advanced scheduling capabilities such as budget-based and group- based scheduling, scheduling applications that have deadlines, custom policies, and advance reservation.

4. BOINC

- Berkeley Open Infrastructure for Network Computing (BOINC) is framework for volunteer and grid computing.
- It allows us to turn desktop machines into volunteer computing nodes that are leveraged to run jobs when such machines become inactive. BOINC supports job check pointing and duplication.
- BOINC is composed of two main components: the BOINC server and the BOINC client.
- The BOINC server is the central node that keeps track of all the available resources and scheduling jobs.
- The BOINC client is the software component that is deployed on desktop machines and that creates the BOINC execution environment for job submission.

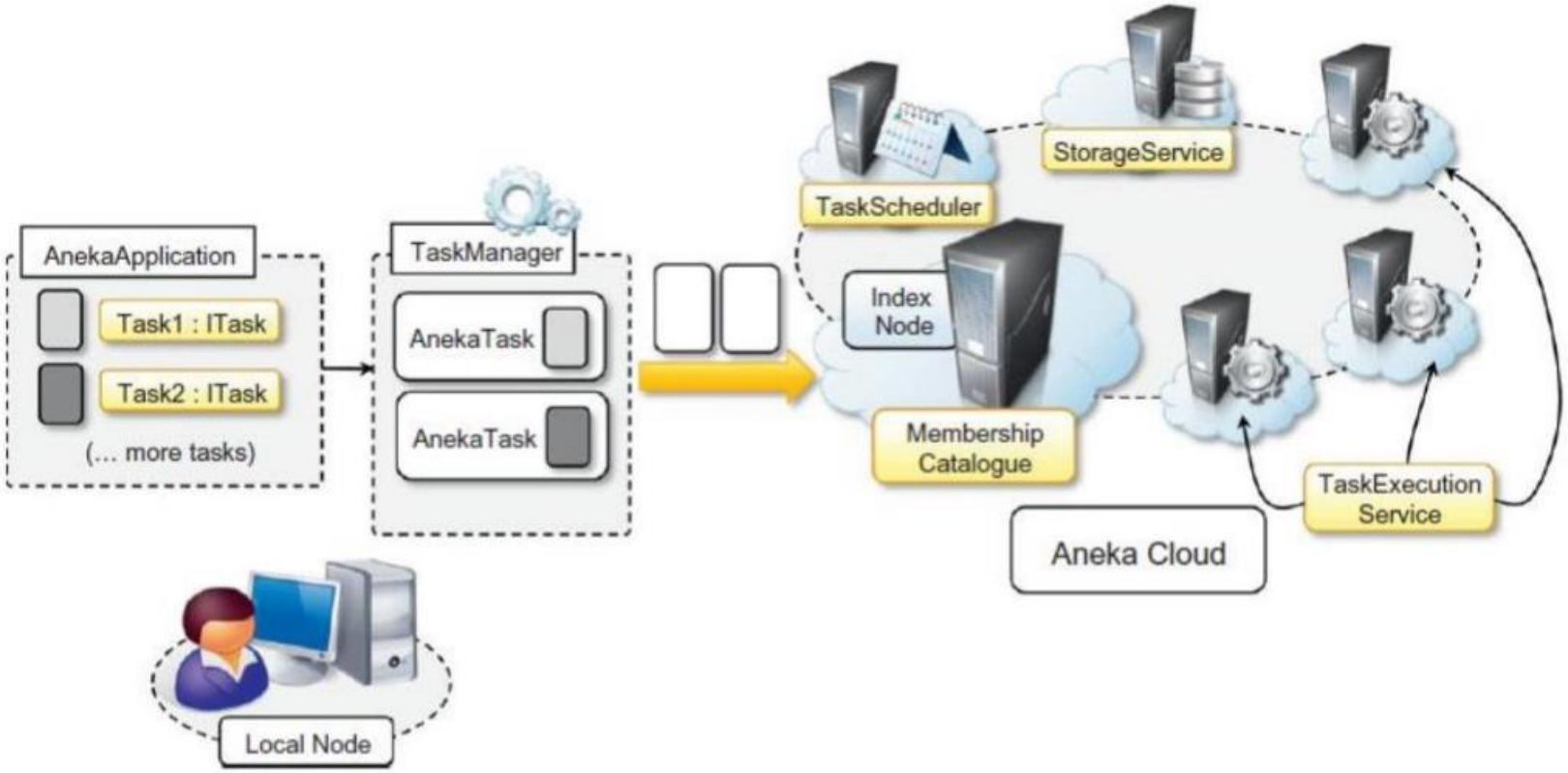
- BOINC systems can be easily set up to provide more stable support for job execution by creating computing grids with dedicated machines.
- When installing BOINC clients, users can decide the application project to which they want to donate the CPU cycles of their computer. Currently several projects, ranging from medicine to astronomy and cryptography, are running on the BOINC infrastructure.

Aneka task-based programming

- Aneka is a software platform for developing cloud computing applications.
- Aneka is a pure PaaS solution for cloud computing.
- Aneka is a cloud middleware product that can be deployed on a heterogeneous set of resources: Like: a network of computers, a multi core server, data centres, virtual cloud infrastructures, or a mixture of all
- The framework provides both middleware for managing and scaling distributed applications and an extensible set of APIs for developing them
- Aneka provides support for all the flavors of task-based programming by means of the Task Programming Model, which constitutes the basic support given by the framework for supporting the execution of bag-of-tasks applications.

Task programming model

- The Task Programming Model provides a very intuitive abstraction for quickly developing distributed applications on top of Aneka.
- It provides a minimum set of APIs that are mostly centered on the Aneka interface.
- Figure provides an overall view of the components of the Task Programming Model and their roles during application execution.


AnekaApplication
Task1 : ITask
Task2 : ITask
(... more tasks)
TaskManager
AnekaTask
AnekaTask
Local Node
StorageService
TaskScheduler
Index Node
Membership Catalogue
TaskExecution Service
Aneka Cloud

- Developers create distributed applications in terms of ITask instances, the collective execution of which describes a running application.
- These tasks, together with all the required dependencies (data files and libraries), are grouped and managed through the Aneka Application class, which is specialized to support the execution of tasks.
- Two other components, AnekaTask and Task Manager, constitute the client-side view of a task-based application.
- The former constitutes the runtime wrapper Aneka uses to represent a task within the middleware; the latter is the underlying component that interacts with Aneka, submits the tasks, monitors their execution, and collects the results.

- In the middleware, four services coordinate their activities in order to execute task-based applications.
- These are: MembershipCatalogue, TaskScheduler, ExecutionService, and StorageService.
- MembershipCatalogue constitutes the main access point of the cloud and acts as a service directory to locate the TaskScheduler service that is in charge of managing the execution of task-based applications.
- Its main responsibility is to allocate task instances to resources featuring the Execution Service for task execution and for monitoring task state.
-

Advantages of using the Task Programming Model

Here are a few advantages of using the Task Programming Model:

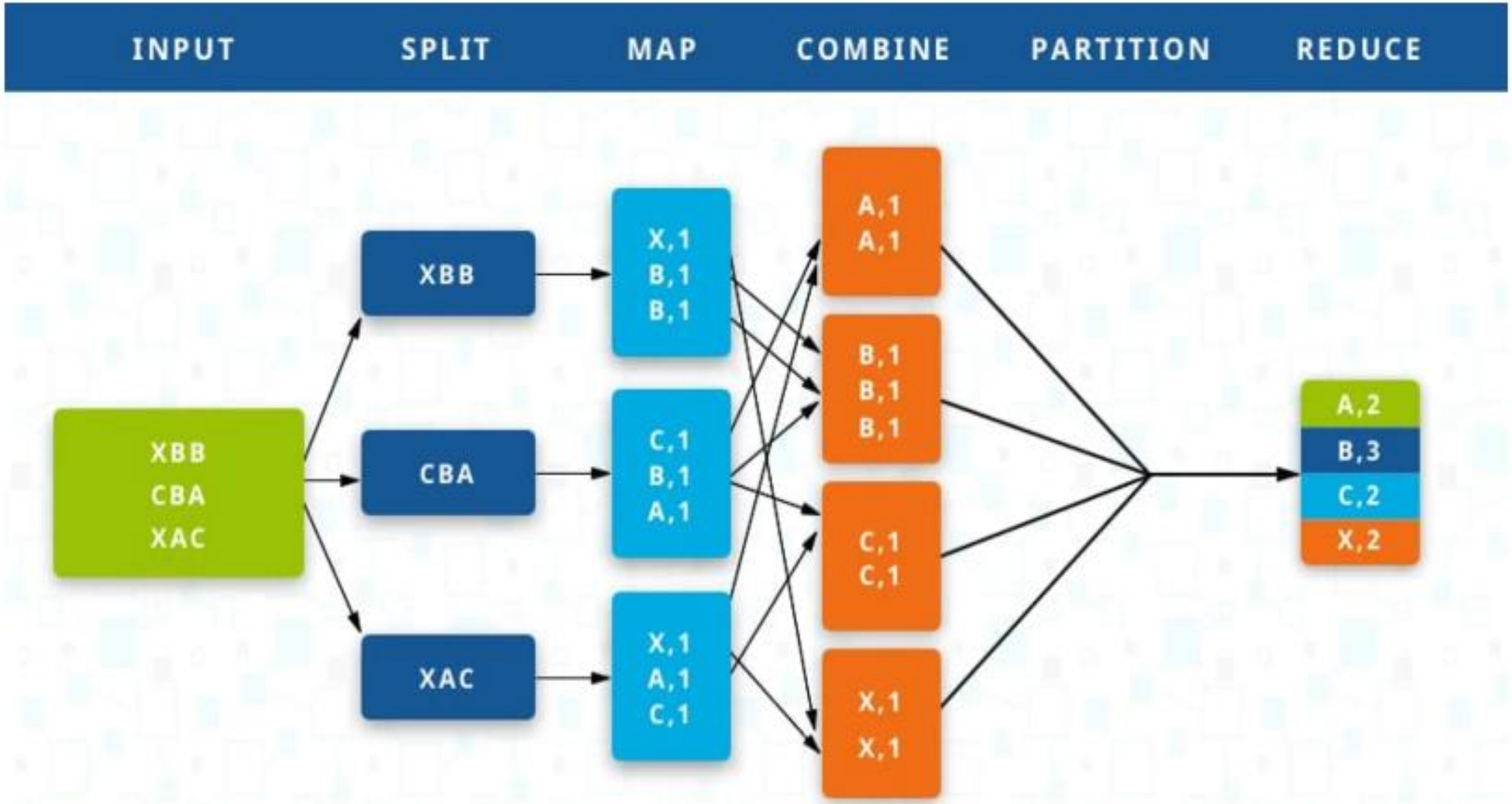
- No explicit threading, users create Tasks, not Threads.
- Maple schedules the Tasks to the processors so that the code scales to the number of available processors.
- Multiple algorithms written using the Task Programming Model can run at the same time without significant performance impact.
- Complex problems can be solved without requiring traditional synchronization tools such as Mutexes and Condition Variables.
- If such synchronization tools are not used, the function cannot be deadlocked.
- The Task functions are simple and the model mirrors conventional function calling.

Map Reduce Programming

- MapReduce is a programming model or pattern within the Hadoop framework that is used to access big data stored in the Hadoop File System (HDFS).
- It is a core component, integral to the functioning of the Hadoop framework.
- MapReduce facilitates concurrent processing by splitting petabytes of data into smaller chunks, and processing them in parallel on Hadoop commodity servers.
- In the end, it aggregates all the data from multiple servers to return a consolidated output back to the application.

- With MapReduce, rather than sending data to where the application or logic resides, the logic is executed on the server where the data already resides, to expedite processing.
- Data access and storage is disk-based—the input is usually stored as files containing structured, semi-structured, or unstructured data, and the output is also stored in files.
- MapReduce was once the only method through which the data stored in the HDFS could be retrieved, but that is no longer the case.

- MapReduce are two functions: Map and Reduce.
- They are sequenced one after the other.
- The **Map** function takes input from the disk as <key,value> pairs, processes them, and produces another set of intermediate <key,value> pairs as output.
- The **Reduce** function also takes inputs as <key,value> pairs, and produces <key,value> pairs as output.


INPUT
SPLIT
MAP
COMBINE
PARTITION
REDUCE
XBB
CBA
XAC
XBB
CBA
XAC
X,1
B,1
B,1
C,1
B,1
A,1
X,1
A,1
C,1
A,1
A,1
B,1
B,1
B,1
C,1
C,1
X,1
X,1
A,2
B,3
C,2
X,2

- Mappers and Reducers are the Hadoop servers that run the Map and Reduce functions respectively.
- It doesn’t matter if these are the same or different servers.
- Mappers and Reducers are the Hadoop servers that run the Map and Reduce functions respectively.
- It doesn’t matter if these are the same or different servers.

Map

- The input data is first split into smaller blocks. Each block is then assigned to a mapper for processing.
- For example, if a file has 100 records to be processed, 100 mappers can run together to process one record each.
- Or maybe 50 mappers can run together to process two records each.
- The Hadoop framework decides how many mappers to use, based on the size of the data to be processed and the memory block available on each mapper server.

Reduce

- After all the mappers complete processing, the framework shuffles and sorts the results before passing them on to the reducers.
- A reducer cannot start while a mapper is still in progress.
- All the map output values that have the same key are assigned to a single reducer, which then aggregates the values for that key.

Parallel Efficiency of Map Reduce

- Parallel efficiency in MapReduce refers to how well a parallel algorithm can make use of multiple computers to solve a problem quickly.
- It depends on factors like dividing the data properly, designing efficient functions, minimizing network delays, balancing the workload, and being able to handle larger amounts of data and computers.
- By measuring metrics like speedup and efficiency, we can see how effective the parallel processing is compared to doing the task sequentially.
- To achieve high parallel efficiency in MapReduce, we need to plan the algorithm carefully, divide the data well, use efficient functions, reduce delays in communication, balance the work between computers, and handle more data and computers effectively.

The parallel efficiency of MapReduce is influenced by several factors:

- **Data Partitioning**: In MapReduce, data is divided into smaller chunks and processed independently. Effective data partitioning ensures that the workload is evenly distributed across the computing nodes, allowing for efficient parallel processing.

- **Map and Reduce Functions**: The efficiency of the map and reduce functions themselves affects parallel efficiency. These functions should be designed to minimize computational overhead and maximize concurrency, enabling multiple tasks to be executed simultaneously.

- **Network Overhead:** MapReduce involves data shuffling and communication between the mapper and reducer nodes. Excessive network overhead can reduce parallel efficiency. Minimizing data movement and optimizing network bandwidth utilization are essential for better parallel efficiency.

- **Load Balancing**: Uneven workload distribution among the nodes can lead to inefficient resource utilization. Proper load balancing techniques, such as dynamic task allocation and workload-aware scheduling, help maintain a balanced workload across the computing resources, improving parallel efficiency.

- **Scalability:** The ability of the MapReduce system to handle increasing data sizes and computing resources affects parallel efficiency. A scalable system can efficiently distribute and process larger volumes of data across a growing number of nodes.

Enterprise batch processing using Map-Reduce

- Enterprise batch processing using MapReduce is a method of efficiently processing large volumes of data in enterprise environments.
- It involves breaking down the data into smaller parts, processing them in parallel on multiple computers, and then combining the results to obtain the desired output.
- Here is a simplified explanation of the steps involved in enterprise batch processing using MapReduce:
- **Data Division:** The input data is divided into smaller chunks, allowing them to be processed independently. This division helps distribute the workload across multiple computers, enabling parallel processing.
- **Mapping**: Each computer applies a mapping function to the data it has received. The mapping function transforms the input data into intermediate key-value pairs.

- **Shuffle and Sort**: The intermediate key-value pairs are then rearranged and sorted based on their keys. This step prepares the data for the next stage of processing.

- **Reducing**: Computers perform a reducing function on the sorted data. The reducing function combines the values associated with each key, generating the final output.

- **Output Aggregation**: The output from the reducing stage is collected and combined to produce the desired output for the batch processing task.

Enterprise batch processing using MapReduce offers several benefits:

- **Scalability**: By distributing the workload across multiple computers, MapReduce allows for the processing of large datasets in a timely manner. Additional computers can be added to the cluster as needed to handle increasing data volumes.
- **Fault Tolerance**: MapReduce is designed to handle failures of individual computers. If a computer fails during processing, the framework redistributes the workload to other computers, ensuring that the task is completed without data loss.
- **Efficiency:** Parallel processing in MapReduce significantly reduces the processing time compared to sequential processing. It leverages the power of multiple computers to process data in parallel, thereby improving overall efficiency.
- **Flexibility**: MapReduce can be applied to various types of batch processing tasks, such as data transformation, data aggregation, and analytics. It provides a flexible framework that can accommodate different enterprise needs.

Comparison between Threads, Task and MapReduce

- Threads, tasks, and MapReduce are different approaches to parallel processing, each with its own characteristics and use cases. Let's compare them:

Granularity:

- Threads: Threads represent the smallest unit of execution within a program. They allow for concurrent execution within a single process and provide fine-grained parallelism.
- Tasks: Tasks represent units of work that can be scheduled and executed independently. They provide a higher level of abstraction compared to threads and are often managed by a task scheduler or executor.
- MapReduce: MapReduce operates on a higher level of abstraction, focusing on dividing data into smaller chunks, processing them independently, and then aggregating the results. It provides a coarse-grained parallelism model.

Data Sharing and Communication:

- Threads: Threads within the same process share memory space, allowing for direct data sharing and communication through shared variables. However, this requires careful synchronization to avoid data conflicts and race conditions.
- Tasks: Tasks can communicate and share data through message passing or by explicitly passing data between tasks. They are designed to minimize data dependencies and facilitate isolated processing.
- MapReduce: In MapReduce, data sharing and communication occur through intermediate key-value pairs. The map phase produces these pairs, which are then shuffled and sorted before being processed by the reduce phase.

Scalability:

- Threads: Scaling threaded applications across multiple machines can be challenging due to issues like data sharing, load balancing, and synchronization. Threads are typically limited to scaling within a single machine with shared memory.
- Tasks: Task-based parallelism can be scaled across multiple machines by distributing tasks to different nodes. This can be achieved using task scheduling frameworks or distributed computing platforms.
- MapReduce: MapReduce is specifically designed for scalability and distributed computing. It can handle large datasets by distributing the workload across a cluster of machines, allowing for horizontal scaling.

Fault Tolerance:

- Threads: Thread-based parallelism does not provide built-in fault tolerance mechanisms. Programmers need to implement error handling and recovery mechanisms to deal with thread failures or exceptions.
- Tasks: Task-based parallelism can incorporate fault tolerance mechanisms through fault detection, error recovery, and task rescheduling. Task schedulers or distributed computing frameworks often include such features.
- MapReduce: MapReduce frameworks, like Hadoop, provide built-in fault tolerance. If a node fails during processing, the framework redistributes the tasks to other nodes, ensuring fault tolerance and completion of the job.

Use Cases:

- Threads: Threads are commonly used in scenarios that require fine-grained control, shared memory access, and synchronization, such as concurrent programming, real-time applications, and GUI programming.
- Tasks: Task-based parallelism is suitable for scenarios that involve independent units of work, such as parallelizing computations, asynchronous programming, and task-driven systems.
- MapReduce: MapReduce is well-suited for large-scale data processing, batch processing, and analytical tasks. It is often used in scenarios like big data processing, data mining, log analysis, and distributed computing.